

Arquitetura do sistema P2P de comunicação (v0.1)

Oct 6, 2026 · @Rodrigo Marques Barbosa

Resumo e princípios

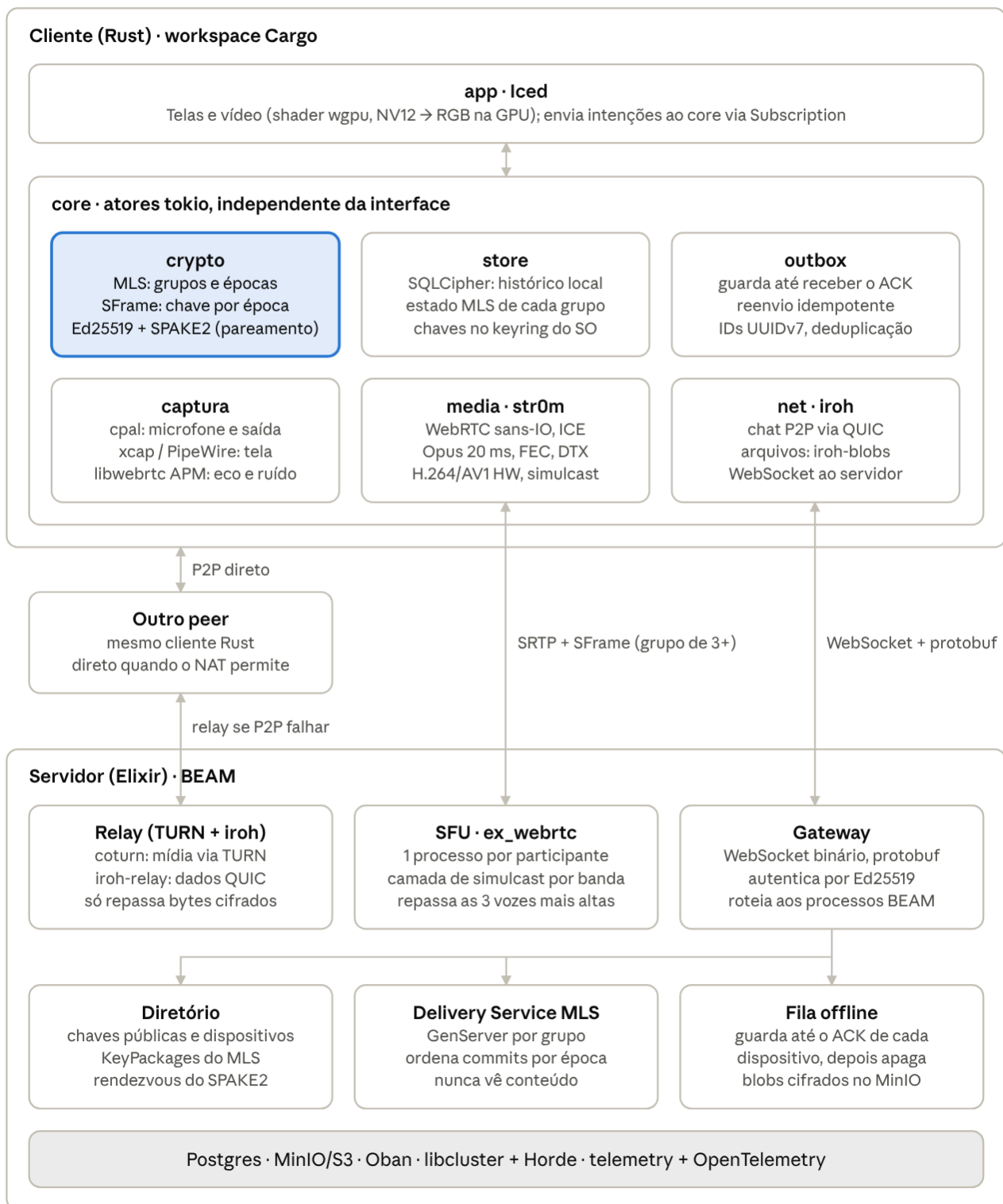
O sistema oferece chat (texto, imagem, vídeo, áudio, arquivo), chamadas de voz e vídeo e espelhamento de tela, em grupo desde a v0.1. O cliente é Rust com Iced; o servidor é Elixir e nunca vê conteúdo. Tudo se apoia em cinco princípios:

1. **Toda conversa é um grupo MLS, inclusive a de 2 pessoas.** Um único modelo de criptografia, sem código separado para 1:1 e grupo.
2. **Tudo é evento dentro do grupo.** Mensagem, reação, edição, confirmação de leitura, convite de chamada e mudança de membros são eventos criptografados.
3. **Mídia criptografada no cliente com SFrame, com chaves derivadas do MLS.** P2P direto ou SFU vira detalhe de transporte; uma chamada de 2 vira grupo sem renegociar criptografia.
4. **O servidor nunca vê conteúdo.** Ele ordena, guarda, repassa e esquece.
5. **Tráfego separado pela natureza.** Mídia em tempo real tolera perda; dados precisam ser confiáveis. Cada um usa a pilha especializada nele.

Visão geral

Conteúdo sai cifrado do cliente; servidor e relays só veem bytes opacos

Cada caixa é um componente; setas mostram o tráfego entre eles; destaque = criptografia



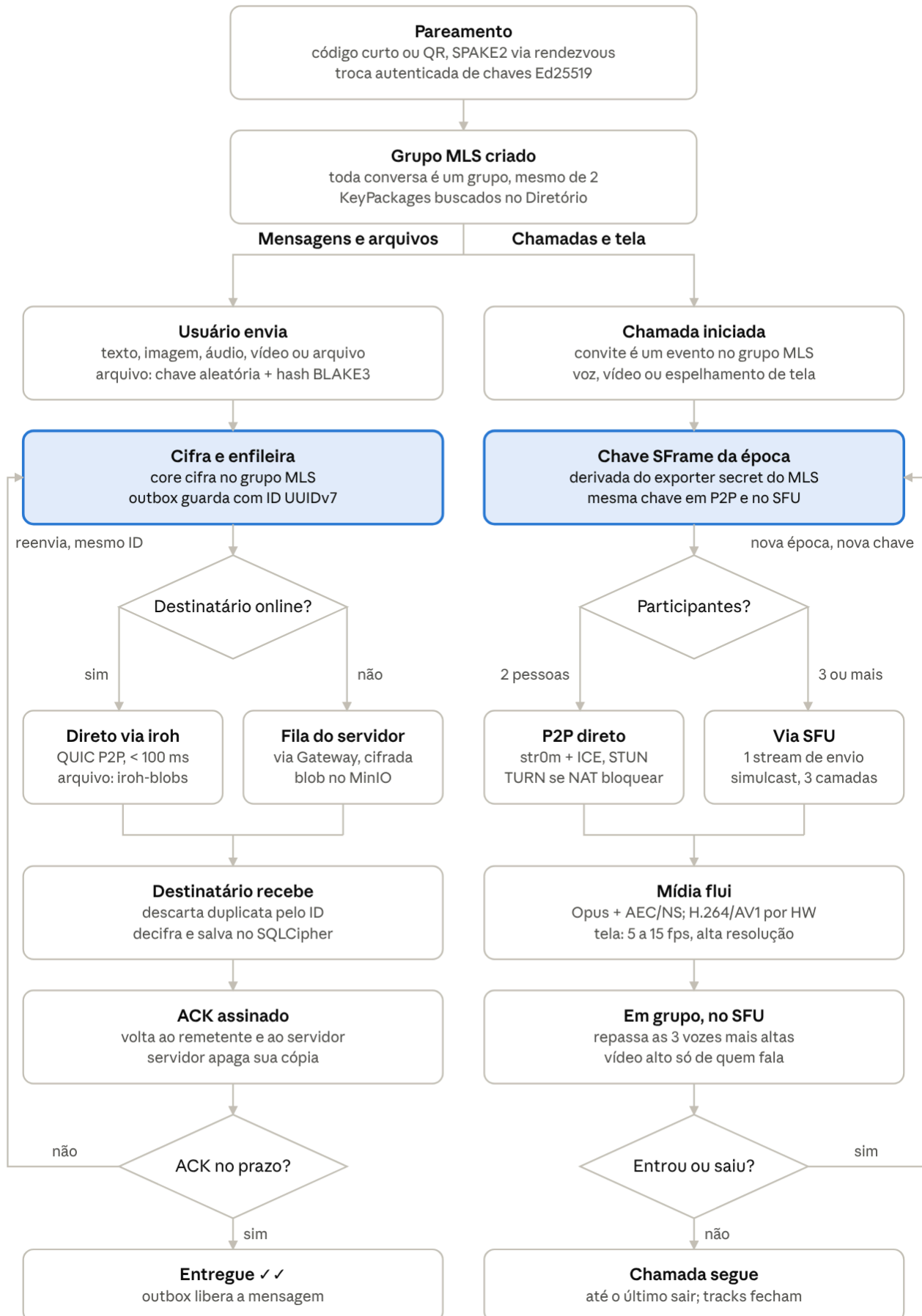
arquitetura · cliente Rust, servidor Elixir, peers

O cliente fala direto com outros peers sempre que pode; o servidor entra para entrega offline, ordenação MLS, grupos via SFU e relay quando o NAT bloqueia.

Fluxo completo

Mensagens esperam ACK; chamadas trocam de rota sem renegociar a criptografia

Do pareamento à entrega: mensagens à esquerda, chamadas à direita; destaque = cifragem



fluxo completo · mensagens e chamadas

Uma mensagem sem ACK volta ao outbox e é reenviada com o mesmo ID; quando alguém entra ou sai de uma chamada, o MLS gera nova época e a rota é reavaliada.

Identidade e pareamento

Cada dispositivo tem sua própria chave; o usuário é o conjunto dos seus dispositivos, assinado por uma chave de identidade.

- **Chaves de dispositivo:** Ed25519, guardadas no chaveiro do sistema operacional (crate `keyring`).
- **Adicionar contato:** código curto ou QR code com SPAKE2 (crate `spake2`), no estilo Magic Wormhole. O servidor só serve de ponto de encontro; as chaves públicas trocadas são autenticadas sem confiar nele.
- **Vincular novo dispositivo:** QR code exibido no dispositivo já logado, com o mesmo mecanismo.
- **Número de segurança:** impressão digital das chaves visível para verificação manual entre contatos.

Criptografia

MLS protege mensagens e chaves de grupo; SFrame protege a mídia com chaves derivadas do MLS.

- **MLS (RFC 9420)** com `openmls`. Entrada e saída de membros custam $O(\log n)$, com forward secrecy e post-compromise security nativos.
- **SFrame (RFC 9605)** para áudio e vídeo, com a chave tirada do *exporter secret* do MLS. Quando alguém sai, o MLS avança de época, a chave do SFrame troca junto e quem saiu deixa de entender a chamada.
- **Arquivos:** chave aleatória por arquivo e endereçamento por hash BLAKE3. Chave e hash viajam dentro da mensagem MLS.
- **Armazenamento local:** SQLite com SQLCipher (`rusqlite` com `bundled-sqlcipher`), chave do banco no chaveiro do sistema.

Limite conhecido: o cabeçalho RTP de nível de áudio (RFC 6464) fica visível ao SFU para detectar quem fala. O SFU sabe quem fala e quando, nunca o que é dito — a mesma escolha de Signal e Google Meet.

Tráfego de dados: iroh

O iroh entrega em uma pilha só QUIC, hole punching, port mapping (UPnP/PCP), IPv6 e relay de fallback.

- **Chat P2P** quando o destinatário está online — a rota mais rápida.
- **Arquivos via `iroh-blobs`**: transferência verificada em streaming (BLAKE3), vinda de várias fontes ao mesmo tempo. Num grupo, quem já baixou ajuda a distribuir; o servidor é só mais uma fonte.
- **Destinatário offline**: a mensagem segue pelo servidor (ver *Servidor Elixir*).

Tráfego de mídia: WebRTC

Mídia usa WebRTC via `str0m` no cliente; o processamento de áudio é o componente que mais define a qualidade percebida.

- **Transporte**: `str0m`, WebRTC sans-IO, sem runtime escondido. Você controla threads e latência; estimativa de banda embutida.
- **Processamento de áudio (essencial)**: `webrtc-audio-processing`, bindings do libwebrtc para cancelamento de eco, supressão de ruído e controle de ganho.
- **Áudio**: `cpal` para captura e reprodução; Opus em quadros de 20 ms com FEC e DTX.
- **Câmera**: H.264 por hardware (VideoToolbox, NVENC, VAAPI, Media Foundation) via `ffmpeg-next`; fallback `openh264`. Simulcast em 3 camadas.
- **Tela**: AV1 ou VP9 com ferramentas de conteúdo de tela quando houver hardware; senão H.264. Resolução alta, 5 a 15 fps.
- **Captura de tela**: `xcap` no Windows, macOS e X11. No Wayland, obrigatoriamente PipeWire + xdg-desktop-portal (crate `ashpd`).

Topologia das chamadas

Duas pessoas falam direto; a partir de três, a mídia passa pelo SFU.

Participantes	Rota	Motivo
2	P2P direto, TURN de fallback	Latência mínima
3 ou mais	SFU	Upload constante: 1 stream enviado, não N-1
Espelhamento para vários	SFU	Um emissor para muitos

A troca de rota é transparente: quando a terceira pessoa entra, os clientes conectam ao SFU, migram os tracks e fecham o P2P. Como o SFrame já está ativo, nada é renegociado na criptografia.

No SFU, cada receptor recebe a camada de simulcast que sua banda suporta. O áudio repassado é o dos 3 mais altos; o vídeo em alta só de quem fala ou está fixado, os demais em miniatura.

Chat efêmero da chamada

Cada sala de chamada tem um chat que vive só na memória dos participantes conectados: quem está na sala guarda e serve o histórico, e quando o último sai, o chat deixa de existir. O servidor nunca guarda nada dele.

Ciclo de vida:

1. **Primeiro a entrar:** cria o log do chat em RAM, sem gravar no SQLCipher.
2. **Mensagens:** difundidas entre os presentes via `iroh-gossip`, cifradas na época MLS da chamada. Anexos vão por `iroh-blobs`, servidos por quem já tem o arquivo.
3. **Alguém entra:** pede o histórico a um peer presente (o de menor latência) por um stream iroh direto. Quanto mais gente na sala, mais fontes servindo o chat.
4. **Alguém sai:** os demais continuam servindo; nada se perde enquanto houver pelo menos um conectado.
5. **Último sai:** a memória é liberada e o chat some.

Decisões de projeto:

- **Histórico para quem chega:** por causa da forward secrecy do MLS, quem entra não consegue decifrar mensagens de épocas anteriores. A solução é o peer que serve o histórico reenviá-lo cifrado na época nova, direto para quem chegou.

- **Integridade:** cada mensagem é assinada pelo autor (Ed25519) e encadeada por hash BLAKE3. Quem recebe o histórico de um peer consegue verificar que nada foi alterado ou omitido.
- **Ordem:** relógio lógico híbrido (HLC) mais o ID UUIDv7 como desempate, para todos verem a mesma ordem sem servidor.
- **Sem ACK nem outbox:** ao contrário das mensagens normais, este chat não tem entrega garantida a quem está offline. Isso é intencional.
- **Limite de memória:** cada cliente guarda no máximo um teto configurável (padrão sugerido: 5.000 mensagens ou 200 MB de anexos); acima disso, descarta os itens mais antigos.
- **Rede ruim:** se a conexão direta falhar, o tráfego passa pelo iroh-relay, que só vê bytes cifrados.

Risco aceito: se todos caírem ao mesmo tempo (queda de rede geral), o chat se perde, mesmo que a intenção fosse continuar a conversa.

Servidor Elixir

Cada conversa e cada chamada é um processo BEAM, supervisionado e distribuído pelo cluster.

- **Gateway:** um WebSocket binário por dispositivo, mensagens em protobuf (`prost` no Rust, `protobuf` no Elixir, mesmo `.proto`). Phoenix, ou `Bandit` + `WebSock` puro, mais enxuto para protocolo binário.
- **Delivery Service do MLS:** um GenServer por grupo ordena os *commits*, garantindo que todos concordem com a ordem das épocas sem locks.
- **Fila de entrega:** pelo menos uma vez, com IDs UUIDv7 e deduplicação no cliente — na prática, exatamente uma vez. A mensagem fica no Postgres até o ACK de cada dispositivo e então é apagada. O ban cuida de expiração e limpeza.
- **Blobs offline:** arquivos criptografados em MinIO ou S3 até todos confirmarem o recebimento.
- **Diretório:** chaves públicas e *KeyPackages* do MLS, para adicionar alguém offline a um grupo.
- **SFU:** `ex_webrtc` , um processo por participante; se um trava, só ele cai.
- **TURN:** coturn, mais relay do iroh hospedado por você.
- **Infraestrutura:** `libcluster` , `Horde` (registro distribuído de processos), `telemetry` e OpenTelemetry desde o primeiro dia.

Trade-off a monitorar: se o SFU em `ex_webrtc` virar gargalo de CPU, a saída é um SFU em Rust com `str0m` , controlado pelo Elixir. A interface entre os dois é pequena.

Cliente Rust

O cliente é um workspace Cargo em que a interface só exhibe estado e envia intenções.

Crate	Responsabilidade
<code>proto</code>	Tipos gerados do protobuf
<code>crypto</code>	openmls, SFrame, identidade
<code>net</code>	iroh e cliente WebSocket
<code>media</code>	Captura, codecs e str0m, em threads dedicadas de tempo real (nunca no tokio)
<code>core</code>	Atores que orquestram tudo; não sabe que a interface existe
<code>app</code>	Iced; conversa com o <code>core</code> via <code>Subscription</code>

Vídeo no Iced: widget `shader` (wgpu), com os planos YUV/NV12 do decoder enviados direto como texturas e conversão para RGB na GPU. Sem conversão na CPU, 9 ou mais vídeos rodam sem esforço.

Com o `core` independente da interface, ele pode ser compilado depois para mobile (interface nativa via `uniffi`) ou para web.

Metas de qualidade

Métrica	Meta	Como atingir
Áudio boca a ouvido	< 150 ms	Opus 20 ms, jitter buffer adaptativo
Início de chamada 1:1	< 1 s	Candidatos ICE pré-coletados com o app aberto
Mensagem P2P	< 100 ms	Conexão iroh já aberta
Mensagem pelo servidor	< 300 ms	WebSocket persistente
Troca de rede (Wi-Fi ↔ 4G)	Sem perda	Migração de conexão QUIC; ICE restart na mídia

Roteiro até a v0.1 e riscos

A v0.1 fecha com chamadas e espelhamento em grupo; cada etapa se apoia na anterior.

1. **Fundação:** identidade, pareamento SPAKE2, MLS e fila de entrega com ACK. Chat de texto ponta a ponta funcionando.
2. **Dados P2P:** iroh para chat direto e `iroh-blobs` para imagens e arquivos, com fallback pelo servidor.
3. **Áudio 1:1:** cpal, processamento de áudio, Opus, `str0m` e SFrame.
4. **Vídeo e tela 1:1:** codificação por hardware, simulcast e renderização no shader do Iced.
5. **Grupo:** SFU no `ex_webrtc`, migração P2P → SFU, seleção de camadas e de quem fala.
6. **Endurecimento:** testes de rede ruim (`tc netem` simulando perda, latência e jitter), métricas e recuperação de falhas.

Riscos principais:

- **Técnico (etapa 3):** eco e qualidade de áudio fazem ou quebram um app de VoIP.
- **De projeto (etapa 1):** se MLS e entrega estiverem certos desde o início, o resto se encaixa por cima.
- **Desempenho do SFU:** `ex_webrtc` pode exigir troca por `str0m` em Rust com muitos participantes.
- **Wayland:** captura de tela só via portal, com permissão do usuário a cada sessão.

Bibliotecas de referência

Lista de memória; confira versões e estado de manutenção antes de fixar dependências.

Camada	Lado	Biblioteca	Uso
Interface	Rust	<code>iced</code>	UI e renderização de vídeo via <code>shader</code>
Runtime	Rust	<code>tokio</code>	Atores do <code>core</code> e rede
Dados P2P	Rust	<code>iroh</code> , <code>iroh-blobs</code>	QUIC, hole punching, relay, arquivos
Mídia	Rust	<code>str0m</code>	WebRTC sans-IO
Áudio	Rust	<code>cpal</code> , <code>audiopus</code> , <code>webrtc-audio-processing</code>	Captura, Opus, eco e ruído
Vídeo	Rust	<code>ffmpeg-next</code> , <code>openh264</code>	Codificação por hardware e fallback
Tela	Rust	<code>xcap</code> , <code>ashpd</code>	Captura; portal no Wayland

Camada	Lado	Biblioteca	Uso
Criptografia	Rust	<code>openmls</code> , <code>sframe</code> , <code>ed25519-dalek</code> , <code>spake2</code> , <code>blake3</code>	MLS, mídia, identidade, pareamento, hashes
Armazenamento	Rust	<code>rusqlite</code> (SQLCipher), <code>keyring</code>	Banco local criptografado, chaves
Protocolo	Ambos	<code>prost</code> / <code>protobuf</code>	Mensagens binárias cliente-servidor
Gateway	Elixir	<code>phoenix</code> ou <code>bandit</code> + <code>websocket</code>	WebSocket binário
Persistência	Elixir	<code>ecto</code> , Postgres, <code>oban</code>	Fila de entrega, diretório, limpeza
SFU	Elixir	<code>ex_webrtc</code>	Chamadas em grupo
Cluster	Elixir	<code>libcluster</code> , <code>horde</code>	Distribuição de processos
Observabilidade	Elixir	<code>telemetry</code> , OpenTelemetry	Métricas e tracing
Relay	Serviço	<code>coturn</code> , <code>iroh-relay</code>	TURN e relay de dados